

EMBEDDED FIRMWARE DESIGN DOCUMENT: SISTEMA DE PREVENCIÓN DE SOBRECALENTAMIENTO PARA DEPORTISTAS

Autores

Samuel Mercado Mercado

Martín Pérez Betancourt

Anna Posada Portilla

María Lucía Quintanilla Álvarez

Al docente

Juan José Londoño Carrillo

En la asignatura

Sistemas Embebidos

Universidad EIA

Envigado

2026

Tabla de contenido

1.	Descripción del funcionamiento del sistema.....	4
1.1.	Alcance del sistema.....	4
1.2.	Características principales	4
2.	Arquitectura de firmware	6
2.1.	Vision general	6
2.2.	Arquitectura del nodo esclavo	7
2.2.1.	Diagrama de bloques	7
2.2.2.	Estructura modular	7
2.2.3.	Tareas del sistema	8
2.2.4.	Flujo de datos.....	8
2.3.	Arquitectura del nodo maestro	10
2.3.1.	Diagrama de bloques	10
2.3.2.	Estructura modular	10
2.3.3.	Tareas del sistema	11
2.3.4.	Flujo de datos.....	11
2.4.	Justificación.....	13
2.4.1.	Topología maestro-esclavo	13
2.4.2.	Elección de ESP-NOW como protocolo de comunicación	13
2.4.3.	Tareas de FreeRTOS	14
2.4.4.	Arquitectura modular por capas.....	14
3.	Diagrama de estados de firmware.....	15
3.1.	Estados del sistema	15
3.2.	Descripción de los estados del sistema	16
3.3.	Transiciones.....	17
4.	Estrategia de manejo de errores	18
4.1.	Sistema de logging	18
4.2.	Protocolos de mitigación de errores	19
5.	Gestión de comunicaciones.....	20
5.1.	ESP-NOW	20
5.2.	I2C	21

5.3. Bluetooth Low Energy (BLE)	22
6. Estructura de directorios del proyecto y flujo de trabajo del repositorio	23
6.1. Estructura de directorios.....	23
6.2. Roles del equipo en el repositorio	24
7. Test Cases	24
8. SRTM.....	25
9. Anexos	26

1. Descripción del funcionamiento del sistema

1.1. Alcance del sistema

El sistema se compone de dos microcontroladores ESP32 que operan en una topología maestro-esclavo, comunicados de forma inalámbrica mediante el protocolo ESP-NOW.

El nodo esclavo (ESP32-C3-DevKit-M1) adquiere la frecuencia cardíaca del deportista mediante el sensor de fotopletimografía MAX30100, con el que se comunica por I2C. Recibe del nodo maestro el valor de temperatura superficial de la piel vía ESP-NOW y presenta ambas variables en un display LCD 1602.

El nodo maestro (ESP32-DevKit-V4) por su parte adquiere la temperatura superficial de la piel mediante el sensor analógico LM35, leyendo su salida a través del ADC interno. Transmite el valor de temperatura al esclavo vía ESP-NOW y, a partir de los valores de temperatura y BPM recibidos, evalúa las condiciones fisiológicas frente a los umbrales definidos. Ante detección de riesgo de sobrecalentamiento, activa la bomba de enfriamiento mediante una señal PWM (duty cycle 0 % o 100 %) que controla un transistor MOSFET IRLZ44N. Este maestro a su vez gestiona además el sistema de logging con salida por Bluetooth Low Energy (BLE) hacia un dispositivo de monitoreo externo.

1.2. Características principales

Tabla 1. Características principales del sistema

Características	Especificación
Arquitectura	Sistema distribuido con dos microcontroladores ESP32 comunicados por ESP-NOW
Microcontrolador Máster	ESP32-DevKit V4 (modulo ESP32-WROOM)
Microcontrolador Slave	ESP32-C-DevKit-M1 (módulo ESP32-C3)
Frecuencia de muestreo	100 Hz
Variables monitoreadas	Temperatura de piel (°C, LM35) y frecuencia cardíaca (BPM, MAX30100)
Rango de temperatura	30 °C a 42 °C, resolución ± 0.5 °C
Rango de frecuencia cardíaca	40 BPM a 220 BPM
Umbrales de alerta	$T > 38.5$ °C o HR > 180 BPM (sostenido ≥ 10 s)
Tiempo de respuesta del actuador	≤ 2 s desde la detección
Alimentación	Batería LiPo 3.7V + boost converter a 5V, autonomía ≥ 30 min
Interfaz de usuario	Display LCD 16x2 I2C + botones físicos
Protocolos de comunicación	ESP-NOW (inter-ESP), I2C (display), BLE (logs), UART (debug), ADC, PWM

1.3. Componentes principales

Tabla 2. División de bloques y su función

Bloque	Componente	Función
ESP Master	ESP32-DevKit V4 (ESP32-WROOM)	Procesamiento de temperatura, evaluación de estado, control de bomba, logging BLE
Adquisición Master	LM35 (sensor analógico)	Medición de temperatura de piel (10 mV/°C) vía ADC
Actuación Master	Microbomba 5V + MOSFET	Circulación de líquido refrigerante (control PWM)
ESP Slave	ESP32-C-DevKit-M1 (ESP32-C3)	Adquisición de FC, manejo de display, navegación por pantallas
Adquisición Slave	MAX30100 (I2C, 0x57)	Sensor óptico de pulso para frecuencia cardíaca
Interfaz Slave	LCD 16x2 con PCF8574 (I2C, 0x27)	Visualización de variables, estado de conexión, pantalla de pausa
Interacción Slave	Botones físicos	Navegación entre pantallas
Comunicación inter-ESP	ESP-NOW (canal 1, 2.4 GHz)	Intercambio de datos entre Master y Slave sin emparejamiento WiFi
Logging remoto	Bluetooth Low Energy (BLE) desde Master	Envío de logs de inicialización y estado a dispositivo cliente
Alimentación	Batería LiPo 3.7V + boost a 5V	Alimentación común para ambos ESP32 y motor

2. Arquitectura de firmware

2.1. Vision general

El firmware del sistema de prevención de sobrecalentamiento se estructura sobre una arquitectura distribuida de dos nodos independientes que operan de forma paralela y se comunican de manera inalámbrica. Cada nodo ejecuta su propio firmware sobre el framework ESP-IDF con FreeRTOS como sistema operativo en tiempo real, organizando su lógica interna en tareas concurrentes con prioridades diferenciadas según la criticidad de cada función.

La distribución entre de las responsabilidades del sistema entre los nodos responde a una separación clara de las tareas a realizar. El nodo esclavo concentra toda la lógica de adquisición de la señal fisiológica de frecuencia cardiaca y la expone los datos al usuario en la interfaz gráfica, mientras que el maestro centraliza la adquisición de temperatura, la evaluación de condiciones de riesgo, control del actuador de enfriamiento y gestión de los registros de estado del sistema.

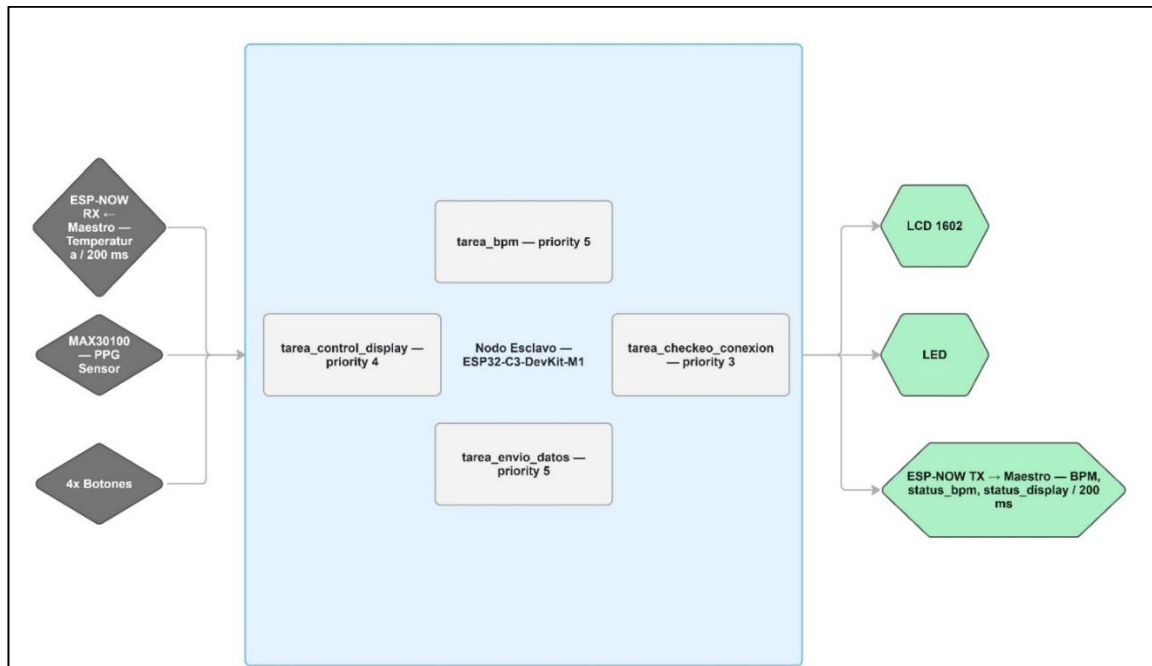
La comunicación entre ambos nodos se realiza mediante ESP-NOW, un protocolo inalámbrico punto a punto que opera usando la capa física de WIFI de cada ESP32 sin la necesidad de requerir de un enrutador ni de la estructura de red circundante. De esta manera, ambos nodos intercambian datos de forma periódica cada 200 ms usando un paquete de datos predefinido que incluye las variables fisiológicas que se han adquirido y el estado operativo de cada uno de sus módulos. Adicionalmente, el nodo maestro, cuenta con una interfaz de logging mediante BLE, permitiendo visualizar en tiempo real el estado de inicialización del sistema y los cambios de estado operativo desde un dispositivo externo.

Por último, el sistema implementa una máquina de estados finita, que reside en el maestro, que evalúa constantemente los valores de temperatura superficial y frecuencia cardiaca recibidos, para así determinar el estado fisiológico del deportista, y decide se activa o desactiva el mecanismo de enfriamiento de forma automática mediante una señal PWM.

2.2. Arquitectura del nodo esclavo

2.2.1. Diagrama de bloques

Figura 1. Diagrama del nodo esclavo



2.2.2. Estructura modular

El firmware del nodo esclavo se organiza a partir de 4 módulos independientes, donde la aplicación principal contenida en el main.c funciona como un orquestador, encargado de inicializar los módulos en orden y crear las tareas en FreeRTOS. La aplicación principal no contiene la lógica funcional principal, sino que esta se encuentra en los módulos. En la siguiente tabla se muestran los módulos que componen al nodo esclavo.

Tabla 3. Módulos del nodo esclavo.

Modulo	Archivo	Responsabilidad
Driver sensor PPG	Max30100	Configuración de registros del MAX30100 por I2C y lectura del FIFO para obtener datos de la señal
Procesamiento de señal PPG	ppg_bpm	Filtrado de señal PPG, detección de picos y cálculo de BPM
Interfaz de usuario	Display	Control del display LCD 1602, gestión de pantallas y

		manejo de interacción del usuario
Comunicaciones	comunicaciones	Configuración de WiFi y ESP-NOW, envío y recepción de paquetes de datos, chequeo del estado de conexión

2.2.3. Tareas del sistema

El nodo esclavo ejecuta 4 tareas concurrentes. La prioridad más alta se asignó a las tareas de adquisición y transmisión de datos por ser las más críticas en tiempo. En la siguiente tabla, se muestran las tareas orquestadas en el main.

Tabla 4. Tareas del sistema del nodo esclavo

Tarea	Prioridad	Periodo (ms)	Responsabilidad
Tarea_bpm	5	10	Lee el fifo del MAX30100 a 100 SPS, procesa la señal PPG y actualiza el BPM local del nodo.
Tarea_envio_datos	5	200	Empaqueta el dato de BPM local y flags de estado, y los envía al maestro por ESP-NOW.
Tarea_control_display	4	100	Actualiza el LCD con el BPM local y temperatura recibida del maestro. Atiende solicitudes de cambio de pantalla.
Tarea_chequeo_conexion	3	5000	Ejecuta un mecanismo de <i>ping pong</i> con el maestro y actualiza el estatus de conexión.

Nota: La repartición de tareas se hizo usando el *task scheduler* de FreeRTOS.

2.2.4. Flujo de datos

El flujo de datos del nodo esclavo parte del sensor MAX30100, cuyo FIFO es leído por *tarea_bpm* cada 10 ms a través del bus I2C. La señal IR cruda, se procesa por el módulo *ppg_bpm*, que aplica la eliminación del componente DC, filtrado pasa-bajas IRR y detección de picos para obtener el valor de BPM, el cual queda almacenado en la variable global *bpm_local*. Cada 200 ms *tarea_envio_datos* toma ese valor, lo empaqueta junto con las banderas de estado del sistema en una estructura *datos_t* y lo transmite al nodo maestro usando ESP-NOW.

En sentido inverso, los paquetes recibidos del maestro, que contienen el valor de temperatura superficial medido localmente por ese nodo, son procesados por el callback de recepción de datos de ESP-NOW, que los almacena para su consulta. La tarea *control_display*, que se ejecuta

cada 100 ms, actualiza tanto el valor de BPM y temperatura, con la información medida y recibida del nodo maestro respectivamente, en el display LCD 1602. Adicionalmente, el nodo cuenta con 4 botones físicos que generan interrupciones que modifican la pantalla activa, permitiendo al usuario navegar entre la vista de datos biométricos, el estado conexión de ESP-NOW, el control manual del PWM y la pantalla de pausa.

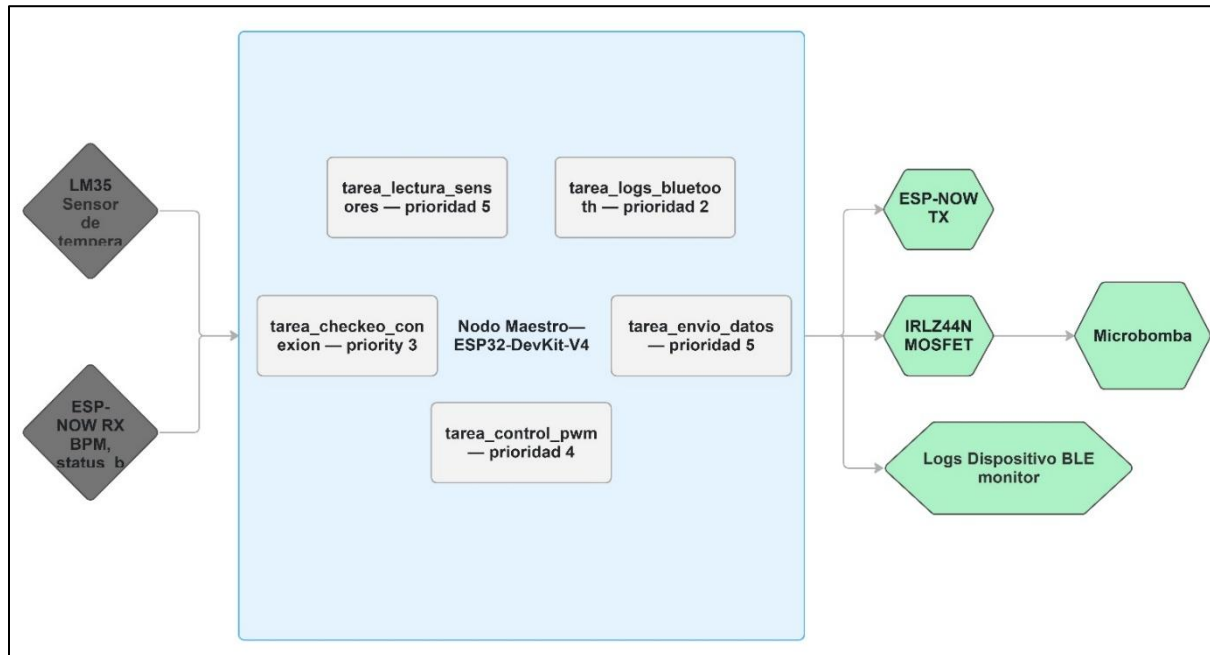
Tabla 5. Síntesis del flujo de datos (esclavo)

Etapa	Descripción	Frecuencia / medio	Resultado
Adquisición	El nodo lee la señal infrarroja del sensor MAX30100 desde su FIFO.	Cada 10 ms por I2C.	Obtiene la muestra cruda del pulso.
Procesamiento	La señal es tratada por el módulo ppg_bpm, que elimina el componente DC, aplica filtrado digital y detecta picos.	En cada ciclo de adquisición.	Calcula el valor de BPM.
Almacenamiento local	El BPM estimado se guarda en la variable global bpm_local.	Inmediato al cálculo.	El dato queda disponible para otras tareas.
Comunicación hacia el maestro	La tarea de envío empaqueta el BPM junto con banderas de estado en una estructura datos_t	Cada 200 ms por ESP-NOW.	El maestro recibe el BPM y el estado del nodo.
Recepción desde el maestro	El callback de ESP-NOW recibe la información enviada por el maestro, principalmente temperatura y estados.	Asíncrono, por recepción de paquete.	Los datos quedan disponibles para consulta local.
Interfaz de usuario	La tarea de display muestra BPM local y temperatura recibida en el LCD 1602.	Cada 100 ms.	El usuario visualiza el estado biométrico y del sistema.
Navegación local	Cuatro botones físicos cambian la pantalla activa mediante interrupciones.	Evento por pulsación.	El usuario alterna entre vistas de datos, conexión y pausa.

2.3. Arquitectura del nodo maestro

2.3.1. Diagrama de bloques

Figura 2. Diagrama del nodo maestro



2.3.2. Estructura modular

El firmware del nodo esclavo se organiza a partir de 4 módulos independientes, donde la aplicación principal contenida en el main.c funciona como un orquestador, encargado de inicializar los módulos en orden y crear las tareas en FreeRTOS. La aplicación principal no contiene la lógica funcional principal, sino que esta se encuentra en los módulos. En la siguiente tabla se muestran los módulos que componen al nodo maestro.

Tabla 6. Módulos del nodo maestro

Módulo	Archivo	Responsabilidad
Adquisición de temperatura	adc_sensores	Configuración del ADC con calibración, timer de muestreo por interrupción y semáforo de sincronización
Control de actuador	pwm	Configuración del canal LEDC y ejecución de rampas suaves de encendido y apagado de la microbomba
Comunicaciones inalámbricas	comunicaciones	Configuración de Wifi y ESP-NOW, envío y recepción de paquetes, mecanismo ping-pong
Logging por BLE	bluetooth_spp	Stack BLE NimBLE con Nordic UART Service. Maneja advertising, conexión, notificaciones y recepción de comandos

2.3.3. Tareas del sistema

El firmware del nodo maestro se organiza a partir de 4 módulos independientes, donde la aplicación principal contenida en el main.c funciona como un orquestador, encargado de inicializar los módulos en orden y crear las tareas en FreeRTOS. La aplicación principal no contiene la lógica funcional principal, sino que esta se encuentra en los módulos.

Tabla 7. Tareas del sistema del nodo maestro

Tarea	Prioridad	Período (ms)	Responsabilidad
tarea_lectura_sensores	5	200	Lee el LM35 por ADC, aplica promedio móvil de 16 muestras, actualiza temperatura_medida y evalúa el umbral de 37 °C
tarea_envio_datos	5	200	Recupera BPM recibido del esclavo, empaqueta datos_t con temperatura y flags de estado, y transmite por ESP-NOW
tarea_control_pwm	4	50	Evalúa flags de alarma y activa o desactiva la microbomba mediante rampa PWM sobre el MOSFET IRLZ44N
tarea_checkeo_conexion	3	5000	Ejecuta ping-pong con el esclavo, actualiza status_conexion y controla el LED indicador en GPIO2
tarea_logs_bluetooth	2	Basada en eventos	Envía logs de inicialización y cambios de estado operativo por BLE. Se activa por notificación de tarea o por comando "logs" recibido por NUS RX

Nota: La repartición de tareas se hizo usando el *task scheduler* de FreeRTOS.

2.3.4. Flujo de datos

El nodo maestro integra dos ramas de datos, una local, basada en la lectura del LM35 mediante el ADC, donde la temperatura se convierte, filtra con promedio móvil y se compara con un umbral para generar la alarma de temperatura; y otra remota, basada en los datos recibidos del nodo esclavo por ESP-NOW, principalmente el BPM y los estados de sus módulos. Ambos flujos convergen en la lógica de control, que evalúa las alarmas cada 50 ms y acciona la microbomba mediante PWM con rampas suaves de encendido y apagado. Además, cualquier cambio de estado se reporta por BLE para supervisión externa.

Tabla 8. Síntesis del flujo de datos (maestro)

Etapa	Descripción	Frecuencia / medio	Resultado
Adquisición	El nodo lee la temperatura mediante un ADC conectado al sensor correspondiente.	Cada 200 ms.	Obtiene el valor bruto de temperatura.
Filtrado y validación	La lectura se convierte a temperatura útil y se suaviza con un promedio móvil.	En cada nueva muestra.	Se obtiene una temperatura más estable.
Evaluación de alarmas	La temperatura procesada se combina con el BPM recibido desde el slave para decidir si existe condición de alarma.	De forma continua.	Se activan o desactivan las condiciones de control.
Control de actuador	Con base en temperatura y BPM, el firmware decide encender o apagar el PWM de la bomba.	Tarea periódica de control.	Se regula el actuador según la lógica del sistema.
Comunicación con el esclavo	El maestro recibe el BPM y el estado del slave, y luego le reenvía una estructura con temperatura, BPM y banderas de estado.	Cada 200 ms por ESP-NOW.	Ambos nodos mantienen datos sincronizados.
Supervisión de enlace	Se ejecuta un ping-pong para verificar la conexión con el slave.	Cada 5 s.	Se actualiza el estado de comunicación.
Reporte externo	El maestro publica logs de inicialización y estado hacia el celular mediante Bluetooth.	Periódicamente, según disponibilidad.	Se obtiene monitoreo externo del sistema.

2.4. Justificación

2.4.1. Topología maestro-esclavo

La decisión de estructurar el sistema a partir de una topología de dos microcontroladores (MCU) independientes reside en una limitación de hardware y de capacidad de cómputo.

En primer lugar, dado que la señal de pulso se adquiere en el dedo y el control del sistema de enfriamiento se encuentra en la zona lumbar, existe un problema para transmitir la señal entre los dos puntos, sin que se comprometa su integridad. La transmisión cableada requiere de componentes y periféricos adicionales que son inconvenientes para un deportista y representan un costo mayor. Por ello, el uso de dos MCU permite hacer el procesamiento de la señal en la región próxima al punto de lectura, eliminando la necesidad de transmisión cableada, al hacer posible la transmisión inalámbrica, y conservando la señal completa.

Esta topología resuelve además un problema de cómputo. El concentrar en un único MCU el procesamiento de la señal PPG, el cálculo BPM con filtrado digital, la comunicación serial con BLE y el control del actuador PWM representaría una carga computacional que puede saturar el procesador y comprometer la ejecución de las tareas más críticas. La distribución de responsabilidades entre nodos permite que el esclavo se dedique exclusivamente a la adquisición de la señal fisiológica, mientras que el maestro concentra el control principal del sistema, garantizándose así que ninguno de los procesadores opere cerca de su límite de capacidad.

Asimismo, esta separación permite definir con mayor claridad las responsabilidades, facilitando el desarrollo, verificación y testeo del sistema.

2.4.2. Elección de ESP-NOW como protocolo de comunicación

La elección de ESP-NOW para la comunicación de ambos MCU radica en dos razones principales. En primer lugar, permite la comunicación inalámbrica punto a punto sin la necesidad de la infraestructura de red (enrutadores) que si requieren plataformas como WiFi. El segundo punto es que al estar integrado en el entorno de ESP-IDF y ser un protocolo propio de Espressif, la configuración del enlace entre los dos nodos es significativamente más sencilla que la de otros protocolos (como bluetooth), reduciendo el tiempo de integración y eliminando la necesidad de depuración constante del canal de comunicación.

Adicionalmente, ESP-NOW ofrece un mecanismo de reconexión automática ante interrupciones del enlace, que en caso de que alguno de los nodos sufra apagado o reinicio, el protocolo restablece la comunicación de forma autónoma una vez que el nodo vuelve a estar disponible,

sin intervención del firmware de la aplicación, lo cual es crítico dado que se requiere un monitoreo constante y que la interfaz comunique de forma confiable la información biométrica, es necesario que el enlace sea estable. Así pues, esta característica del protocolo aumenta la confiabilidad del enlace en un entorno de uso deportivo donde las interrupciones momentáneas de energía o interferencias son escenarios plausibles.

2.4.3. Tareas de FreeRTOS

La organización del firmware alrededor de tareas en FreeRTOS con prioridades diferenciadas responde a la necesidad de atender múltiples funciones con requisitos temporales distintos sin bloquear la ejecución global del sistema. Un ciclo bloqueante secuencial habría introducido latencia en la lectura de los sensores y la transmisión de datos, que son puntos clave del funcionamiento del sistema.

La asignación de prioridades refleja directamente la criticidad de cada función y la exigencia computacional de cada tarea. Las tareas de adquisición de señales fisiológicas y transmisión de datos reciben la prioridad más alta, ya que constituyen el núcleo funcional del sistema y además representan la mayor carga de procesamiento: la lectura del MAX30100 a 100 SPS con filtrado digital y detección de picos, y la lectura del LM35 con promedio móvil de 16 muestras, son operaciones que deben ejecutarse en intervalos estrictos sin ser desplazadas por otras tareas. En el extremo opuesto, tareas como el logging por BLE y el control PWM reciben prioridades menores porque su naturaleza es reactiva, dado que el logging se activa únicamente ante cambios de estado o solicitudes externas, y el control PWM opera sobre una señal de baja dinámica temporal donde un retardo de decenas de milisegundos no compromete el funcionamiento del sistema.

Adicionalmente, la separación en tareas permite simplificar la integración de los módulos y la evaluación de cada uno de ellos, pues en caso de falla o error, la funcionalidad que no se encuentra operativa se relaciona directamente con la tarea que la ejecuta y los módulos de los cuales esta depende, lo que reduce significativamente el espacio de búsqueda durante la depuración y facilita el aislamiento del componente defectuoso.

2.4.4. Arquitectura modular por capas

La organización del firmware en módulos independientes con interfaces definidas responde a varias razones. En primer lugar, existen módulos compartidos entre ambos nodos cuyas estructuras de datos deben coincidir necesariamente, como es el caso del módulo de comunicaciones, que define la estructura *datos_t* y el mecanismo de ping-pong utilizados tanto por el esclavo

como por el maestro. Así pues, el hecho de centralizar esta lógica en un único módulo compartido garantiza la consistencia del protocolo de intercambio entre nodos y elimina la duplicación de código.

En segundo lugar, la separación en módulos introduce una capa de abstracción entre la lógica funcional de la aplicación y la configuración de bajo nivel del hardware. Esta abstracción permite actualizar o modificar la implementación interna de un módulo, por ejemplo, cambiar el esquema de calibración del ADC o el registro de configuración del MAX30100, sin que dichos cambios propaguen roturas hacia el resto del sistema. Cada módulo expone únicamente su interfaz a través del archivo de cabecera correspondiente, y los demás módulos dependen exclusivamente de esa interfaz, no de su implementación.

Finalmente, la ausencia de dependencias cruzadas entre módulos facilita tanto la integración incremental durante el desarrollo como el mantenimiento a largo plazo del código. Cada módulo puede ser desarrollado, probado e integrado de forma independiente, y cualquier modificación futura queda acotada al módulo correspondiente sin riesgo de afectar el comportamiento del resto del sistema.

3. Diagrama de estados de firmware

3.1. Estados del sistema

1. Inicialización.
2. Operación normal
3. Comunicación activa entre nodos.
4. Alarma por temperatura alta.
5. Alarma por BPM alto.
6. Actuación del PWM/bomba.
7. Conexión degradada o perdida.
8. Registro y supervisión BLE en el máster.

3.2. Descripción de los estados del sistema

Tabla 9. Estados del sistema

Estado	Nodo	Descripción
Inicialización	Ambos	Se levantan NVS, Wifi, ESP-NOW y periféricos propios de cada nodo. El master además inicializa BLE y el control PWM; el slave inicializa MAX30100 y display.
Operación normal	Ambos	El slave mide BPM y actualiza su pantalla; el master mide temperatura, recibe BPM del slave y mantiene el intercambio de datos activo.
Comunicación activa entre nodos	Ambos	Existe intercambio periódico por ESP-NOW. El master envía temperatura y estados; el slave devuelve BPM y estado de display.
Alarma por temperatura alta	Master	Se activa cuando la temperatura medida por el master supera el umbral definido. Esta alarma alimenta la lógica de decisión del PWM.
Alarma por BPM alto	Slave y Master	El slave calcula BPM localmente y lo reporta al master. Si el BPM supera el umbral, el master lo toma como condición de alarma.
Actuación del PWM/bomba	Master	El master enciende o apaga suavemente el PWM según la combinación de alarmas y los tiempos de persistencia definidos.
Conexión degradada o perdida	Ambos	Se detecta por ping/pong ESP-NOW. Si no hay respuesta dentro del tiempo esperado, el estado de conexión pasa a falso y se apaga el indicador local.
Registro y supervisión BLE	Master	El master publica por BLE los estados de inicialización y operación para monitoreo externo.

3.3. Transiciones

Tabla 10. Transiciones de estado

Estado origen	Condición de transición	Estado destino	Nodo involucrado
Inicialización	Termina la configuración de NVS, Wifi, ESP-NOW, sensores y actuadores	Operación normal	Ambos
Operación normal	Temperatura medida por el master al umbral	Alarma por temperatura alta	Master
Operación normal	BPM calculado por el slave mayor al umbral	Alarma por BPM alto	Slave y master
Alarma por temperatura alta + Alarma por BPM alto	Ambas alarmas se mantienen activas durante el tiempo mínimo definido	Actuación del PWM/bomba	Master
Actuación del PWM/bomba	Una o ambas alarmas dejan de cumplirse y se vence el tiempo de liberación	Operación normal	Master
Operación normal	No se recibe respuesta válida de ping/pong dentro de la ventana esperada	Conexión degradada o perdida	Ambos
Conexión degradada o perdida	Se restablece el intercambio válido por ESP-NOW	Operación normal	Ambos
Cualquier estado operativo	Cambio en conexión, temperatura, BPM, PWM o display	Registro y supervisión BLE	Master

4. Estrategia de manejo de errores

4.1. Sistema de logging

El sistema de logging está planteado como una interfaz de observación operativa del sistema completo Master - Slave, no como mensajes sueltos de depuración de cada uno de los nodos. Su objetivo principal es mostrar el estado real del equipo en cada fase, desde el arranque, pasando por la operación continua y los cambios de condición. El sistema maneja dos tipos de reporte complementarios: uno de inicialización (que confirma si cada subsistema crítico quedó correctamente habilitado) y otro de estado en ejecución (que refleja salud funcional de comunicación, sensado y actuación).

En la práctica, el Master centraliza esta información y la envía hacia la aplicación de monitoreo por BLE, actuando como punto único de monitoreo del sistema integrado. Esto permite que, aunque físicamente existan dos nodos, desde la interfaz externa se vea una sola captura de estado coherente. El resultado es una trazabilidad más clara para operación y mantenimiento, que hace posible identificar rápidamente si la falla viene del enlace, del censado o de la lógica de control.

Además, el esquema está orientado a eventos, pues el sistema prioriza enviar logs cuando hay un cambio relevante de estado, y no únicamente por temporización fija. Esta decisión mejora la calidad del log porque evita ruido de mensajes repetidos y, al mismo tiempo, reduce carga de CPU y tráfico inalámbrico innecesario. Como respaldo, se mantiene un envío periódico de seguridad para no perder visibilidad si no ocurren cambios durante intervalos largos.

Finalmente, el logging combina dos niveles de uso, hacia usuario/app, entrega reportes operativos legibles y accionables, mientras que, hacia ingeniería, conserva mensajes técnicos de diagnóstico para análisis de fallos de comunicación, suscripción BLE, lecturas inválidas o reinicios de servicios. Esta separación permite que el mismo sistema de logs sirva tanto para supervisión en campo como para depuración en laboratorio durante el diseño.

En la siguiente tabla se muestra las diferentes variables de estado reportadas por el sistema:

Tabla 11. Variables de estado

Salida de log	Significado	Función
INIT: W	Estado de arranque de Wifi (1 ok, 0 falla)	Saber si la base de comunicación inició bien
INIT: E	Estado de arranque de ESP-NOW (1 ok, 0 falla)	Confirmar enlace Master-Slave disponible
INIT: A	Estado de inicialización del sensor/ADC (1 ok, 0 falla)	Verificar que la lectura de sensores está lista
INIT: I	Estado de inicialización de I2C/Display (1 ok, 0 falla)	Confirmar que la interfaz local puede mostrar datos
INIT: P	Estado de inicialización de PWM (1 ok, -1/0 no listo o falla)	Saber si el actuador puede controlarse
PWM	Estado actual de la bomba/actuador PWM (0 apagado, >0 activo)	Ver acción real del sistema ante alarmas
Display	Salud del display (1 ok, 0 falla)	Detectar si la salida visual local está disponible
Conexión	Estado del enlace Master - Slave (1 conectado, 0 desconectado)	Detectar pérdida de comunicación en campo
Temp	Validez de lectura de temperatura (1 válida, 0 inválida)	Saber si la temperatura es confiable
Bpm	Validez de lectura de BPM (1 válida, 0 inválida)	Saber si la señal biométrica es confiable

4.2. Protocolos de mitigación de errores

La mitigación de errores está diseñada con enfoque por capas, buscando continuidad operativa antes que detener completamente el sistema ante cualquier anomalía. En la capa de arranque, se validan inicializaciones críticas y se corrigen condiciones recuperables de memoria no volátil, evitando que el equipo inicie en estados inconsistentes. Con ello, se reduce la probabilidad de fallas encadenadas desde el inicio de operación.

En la capa de comunicación entre nodos, se implementa supervisión activa del enlace mediante intercambio periódico de mensajes de vida y ventanas de tiempo de validez. Si el enlace se degrada o se pierde, el sistema lo detecta por timeout y cambia explícitamente el estado de conexión a falla, reflejándolo en indicadores y en telemetría. Esto evita falsas confirmaciones y permite diagnóstico rápido de caídas de red o interferencia.

En la capa de integridad de datos, los mensajes recibidos se validan antes de procesarse, descartando tramas inesperadas o incompletas para proteger el estado interno. En paralelo, los módulos de censado aplican criterios temporales y de calidad de señal (filtros, umbrales dinámicos y ventanas válidas) para evitar decisiones basadas en ruido o mediciones invalidas.

Cuando la señal no cumple condiciones mínimas, el sistema rechaza ese dato y reporta estado degradado en lugar de producir una salida engañosa.

A nivel de control, la estrategia privilegia degradación segura: ante fallas parciales de sensor, display o comunicación, el sistema sigue ejecutando funciones esenciales y mantiene informado su estado de salud, en vez de colapsar por completo. Esto se complementa con tareas desacopladas y temporización controlada, lo que evita bloqueos por espera activa y mejora robustez temporal. En conjunto, estos protocolos convierten errores transitorios en eventos gestionables y hacen que el comportamiento global sea predecible, observable y recuperable.

5. Gestión de comunicaciones

El sistema utiliza tres mecanismos de comunicación, ESP-NOW para el intercambio principal de datos entre nodos, I2C para el periférico local del esclavo, y BLE con NUS para consulta de logs desde una aplicación externa en el maestro.

5.1. ESP-NOW

ESP-NOW opera como el canal principal de comunicación entre el nodo maestro y el nodo esclavo. Su elección frente a otras alternativas inalámbricas se fundamenta en su capacidad de establecer comunicación punto a punto sin infraestructura de red adicional y en su integración nativa dentro del entorno ESP-IDF, lo que simplifica significativamente la configuración del enlace entre los dispositivos. La tabla a continuación resume los parámetros de configuración del protocolo en el sistema.

Tabla 12. Parámetros de configuración de ESP-NOW

Parámetro	Valor
Topología	Punto a punto
Canal Wifi	1
MACs	Maestro: 20:E7:C8:AC:9E:24 — Esclavo: D8:3B:DA:27:A5:B0
Seguridad	Sin cifrado
Potencia TX	Reducida (80)
Estructura datos (M→S)	temperatura, bpm, status_display, status_temp, status_bpm
Estructura datos (S→M)	bpm, status_display, status_bpm
Período de envío	200 ms

Verificación enlace	Cada 5 s; conexión considerada activa si respuesta dentro de 5 s
Indicador local	LED indica estado de conexión (ON = OK)

5.2. I2C

El protocolo I2C es utilizado exclusivamente en el nodo esclavo, donde funciona como bus compartido entre el sensor MAX30100 y el display LCD 1602, este último conectado a través de un expensor de puertos PCF8574. El bus opera en modo estándar a 100 kHz, velocidad suficiente para las tasas de transferencia requeridas por ambos periféricos, e integrar un semáforo para controlar el acceso al bus. Los resistores de pull-up necesarios para la correcta operación del bus están implementados en hardware, por lo que se encuentran deshabilitados en la configuración por software del driver. La tabla a continuación resume los parámetros de configuración del bus I2C en el sistema.

Tabla 13. Parámetros configuración bus I2C

Parámetro	Valor
Topología	Bus compartido en el nodo esclavo (MAX30100 + LCD vía PCF8574)
Velocidad	100 kHz
Dirección MAX30100	0x57
Dirección LCD (PCF8574)	0x27
Pines físicos	SDA = GPIO0, SCL = GPIO1
Puerto I2C	0
Resistencias pull-up	Deshabilitados en configuración
Uso	MAX30100: FIFO lectura periódica; LCD: visualización BPM/Temp/estado

5.3. Bluetooth Low Energy (BLE)

El canal Bluetooth Low Energy opera exclusivamente en el nodo maestro con fines de monitoreo y logging remoto del sistema. Su implementación se basa en NimBLE de ESP-IDF, exponiendo el Nordic UART Service (NUS), que permite a un evaluador consultar el estado operativo del sistema desde un dispositivo externo sin conexión física. Dado que el MTU por defecto de BLE es de 23 bytes, los mensajes de mayor longitud se fragmentan automáticamente en trozos de aproximadamente 20 bytes. La tabla a continuación resume los parámetros de configuración del canal.

Tabla 14. Parámetros configuración del canal BLE

Parámetro	Valor
Disponibilidad	Nodo maestro
Nombre del dispositivo	ESP32-MASTER
Servicio	Nordic UART Service
Características	RX: escritura (comandos desde cliente) — TX: notificaciones (logs hacia cliente)
Comando soportado	"logs" (solicita envío inmediato de estado)
Formato mensajes	INIT: INIT: W:%d, E:%d, A:%d, I:%d, P:%d — STATUS multiline: PWM, Display, Conexion, Temp,Bpm
Fragmentación	Mensajes se fragmentan en trozos ~20 bytes para MTU por defecto
Modo envío	Notificaciones (si el cliente esta suscrito)

6. Estructura de directorios del proyecto y flujo de trabajo del repositorio

6.1. Estructura de directorios

El proyecto se encuentra alojado en el repositorio público de GitHub Proyecto-SE-2026, donde se mantiene la totalidad del código fuente, las librerías de soporte y la documentación técnica del sistema. La organización del repositorio responde a la arquitectura distribuida del firmware, separando físicamente el código de cada nodo en directorios independientes, de modo que el desarrollo, compilación y verificación de cada uno se realiza de forma aislada sin interferir con el otro. Esta separación facilita además el manejo de las dependencias específicas de cada framework, dado que el nodo maestro emplea el módulo ESP32-WROOM y el nodo esclavo el ESP32-C3, lo que requiere configuraciones distintas de sdkconfig dentro del entorno ESP-IDF.

En la raíz del repositorio se encuentran cuatro directorios principales y un archivo README.md, cuya descripción se presenta en la siguiente tabla.

Tabla 15. Resumen de archivos del repositorio

Directorio / Archivo	Contenido y Propósito
Firmware_master/	Proyecto ESP-IDF del nodo maestro (ESP32-DevKit V4). Contiene el main.c de orquestación y los módulos adc_sensores, pwm, comunicaciones y bluetooth_spp. Incluye los archivos CMakeLists.txt y sdkconfig propios del proyecto.
Firmware_slave/	Proyecto ESP-IDF del nodo esclavo (ESP32-C3-DevKit-M1). Contiene el main.c de orquestación y los módulos MAX30100, ppg_bpm, display y comunicaciones. Incluye los archivos CMakeLists.txt y sdkconfig propios del proyecto.
Librerias/	Componentes y librerías de terceros utilizadas por ambos nodos (drivers de display, utilidades I2C, etc.). Se mantienen separadas del firmware para facilitar su reutilización y actualización independiente.
docs/	Documentación del proyecto: documento de diseño de firmware, esquemático, requisitos del proyecto, matriz SRTM y registros de test cases.
README.md	Descripción general del proyecto, integrantes del equipo con sus respectivos roles, objetivo general y objetivos específicos.

Dentro de cada uno de los proyectos de firmware (Firmware_master y Firmware_slave) se mantiene la estructura estándar definida por ESP-IDF, que incluye un directorio main/ con la lógica principal de la aplicación, un directorio components/ con los módulos internos del nodo y los archivos de configuración del sistema de build basado en CMake. Cada módulo descrito en las secciones 2.2.2 y 2.3.2 (por ejemplo, max30100, ppg_bpm, display, comunicaciones,

adc_sensores, pwm, bluetooth_spp) se implementa como un componente independiente con su propio archivo de cabecera, su archivo fuente y su archivo CMakeLists.txt local, lo cual garantiza la independencia entre módulos definida por la arquitectura modular del firmware.

6.2. Roles del equipo en el repositorio

La asignación de responsabilidades sobre el repositorio se deriva directamente de los roles definidos para el equipo de trabajo. Cada integrante interviene en zonas específicas del repositorio y participa en el flujo de revisión y aprobación de cambios de acuerdo con su rol.

Tabla 16. Responsabilidades del repositorio

Integrante	Rol	Responsabilidades sobre el repositorio
María Lucía Quintanilla Álvarez	Technical Lead	Administración del repositorio, revisión y aprobación de pull requests, integración de los nodos maestro y esclavo, mantenimiento de la rama main.
Samuel Mercado Mercado	Firmware Engineer	Desarrollo de los módulos de adquisición, procesamiento de señal y comunicaciones de ambos nodos.
Martín Pérez Betancourt	Hardware Integration Engineer	Configuración de periféricos a bajo nivel (ADC, PWM, I2C, GPIO), integración del firmware con el hardware físico y depuración sobre el prototipo.
Anna Posada Portilla	Verification & Testing Engineer	Ejecución de los test cases definidos en el SRTM, registro de evidencias y documentación de pruebas en el directorio docs.

7. Test Cases

Con base a los requisitos funcionales y no funcionales previamente establecidos, los tests cases se diseñaron con el propósito de verificar el correcto funcionamiento del sistema tanto a nivel de hardware como de software, validando la adquisición de señales fisiológicas, el procesamiento de datos, la activación y desactivación de actuadores, y la comunicación entre módulos. Asimismo, las pruebas permitieron comprobar la estabilidad del sistema, la correcta integración de protocolos de comunicación, la interacción con el usuario y el desempeño general del prototipo en condiciones de operación reales. Los resultados obtenidos, evidencias y registros completos de ejecución se incluyen en *Anexo_Test_Cases*.

8. SRTM

La Matriz de Trazabilidad de Requisitos (SRTM) vincula cada requisito funcional y no funcional del sistema con el caso de prueba que lo valida, garantizando que toda condición establecida en la fase de diseño cuenta con un mecanismo de verificación asociado.

La SRTM completa se encuentra en docs/SRTM.pdf del repositorio. A continuación, se presenta un extracto resumido para referencia. Para cada requisito se documenta: identificador del requisito, descripción, caso de prueba que lo valida, tipo de prueba aplicada y artefactos de evidencia generados durante su ejecución.

Tabla 17. Resumen de la SRTM

Req ID	Requisito	Test Case ID	Tipo de Test	Artefactos
RF-01	Lectura de temperatura de piel	TCF-01	Integración / Hardware	Captura de display LCD, monitor serial, logs ADC
RF-02	Lectura de frecuencia cardíaca	TCF-02	Integración / Hardware	Logs idf.py monitor, captura BPM, display LCD
RF-03	Detección de condición de sobrecalentamiento	TCF-03	Funcional / Integración	Logs BLE, captura de transición de estado
RF-04	Activación de microbomba refrigerante	TCF-03	HIL / Tiempo real	Osciloscopio PWM, video bomba activa, logs BLE
RF-05	Desactivación de microbomba refrigerante	TCF-04	HIL / Tiempo real	Osciloscopio PWM, logs BLE, video bomba apagándose
RF-06	Sistema de comunicaciones	TCF-05	Integración / Comunicación	Capturas BLE, display funcionando, logs ESP-NOW
RF-07	Módulo de registros y manejo de errores	TCF-05	Funcional / Comunicación	Respuesta BLE NUS, logs INIT y estado
RF-08	Conversión analógica-digital de las señales	TCF-01, TCF-02	Hardware / Integración	Valores ADC, logs seriales, captura sensores
RF-09	Interfaz de usuario	TCF-06	Funcional / UI	Video navegación pantallas, fotos LCD
RF-10	Procesamiento y toma de decisiones	TCF-04	Integración / Firmware	Logs de estado, captura PWM y display
RF-11	Control de alimentación	TCF-07	Hardware / Potencia	Mediciones multímetro, fotos PCB
RNF-01	Implementación física	TCF-07	Inspección / Hardware	Fotografías del prototipo, PCB y carcasa.
RNF-02	Alimentación autónoma	TCF-07	Validación	Video continuo, registros de operación

RNF-03	Tiempo de respuesta del sistema	TCF-03	Tiempo real / Rendimiento	Cronómetro, osciloscopio, logs PWM
RNF-04	Estándares de código y versiones	TCF-08	Revisión técnica	Repositorio GitHub, commits, código fuente
RNF-05	Trazabilidad completa de requisitos	TCF-08	Documentación / Calidad	Documento SRTM, matriz de trazabilidad
RNF-06	Optimización del uso de recursos	TCF-08	Rendimiento / Firmware	Reporte idf.py size, uso RAM/Flash

9. Anexos

Anexo_Test_cases: <https://docs.google.com/document/d/1xHqSYE-Yhnz1nRz2XJ4nIkT9z803QroSd1tCkx6ejhY/edit?usp=sharing>

Anexo_SRTM: [SRTM.xlsx](#)